Object Detection

Use Python to demonstrate **Ultralytics**: Object Detection effects on images, videos, and real-time detection.

1. Model Introduction

Object detection is a task involving identifying the location and category of objects in images or video streams.

The output of an object detector is a set of bounding boxes that surround objects in the image, along with class labels and confidence scores for each box. If you need to identify objects of interest in a scene but don't need to know the specific location or exact shape of the objects, object detection is a good choice.

2. Target Prediction: Image

Use yolo26n.pt to predict images in the ultralytics26 folder.

Enter the code folder:

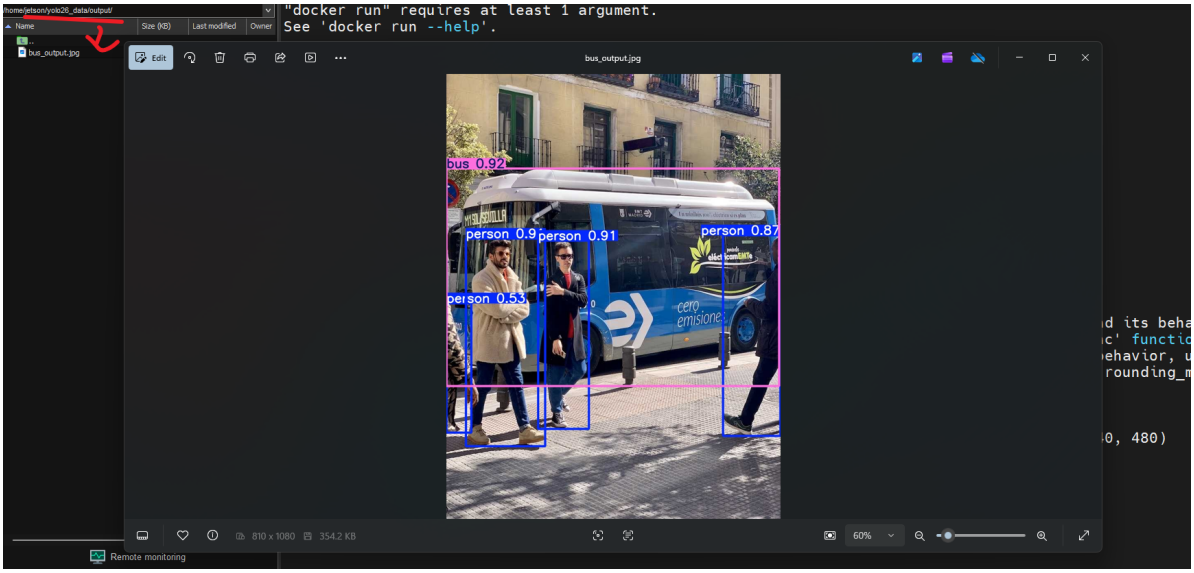
```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 01.detection_image.py
```

Effect Preview

yolo recognition output image location: /root/yolo26_data/output/



Sample code:

```
from ultralytics import YOLO

# Load a model
# model = YOLO("/ultralytics/yolo26n_ncnn_model")
model = YOLO("/ultralytics/yolo26n.pt")

# Run batched inference on a list of images
results = model("/root/yolo26_data/bus.jpg")  # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes  # Boxes object for bounding box outputs
    # masks = result.masks  # Masks object for segmentation masks outputs
    # keypoints = result.keypoints  # Keypoints object for pose outputs
    # probs = result.probs  # Probs object for classification outputs
    # obb = result.obb  # Oriented boxes object for OBB outputs
    result.show()  # display to screen
    result.save(filename="/root/yolo26_data/output/bus_output.jpg")  # save to
disk
```

3. Target Prediction: Video

Use yolo26n_ncnn_model to predict videos in the ultralytics26 folder (not videos that come with ultralytics).

> Note: JetsonNano 4G version may get stuck and restart. In this case, you can switch to a shorter video for recognition

Enter the code folder:

```
cd /ultralytics/yahboom_demo
```

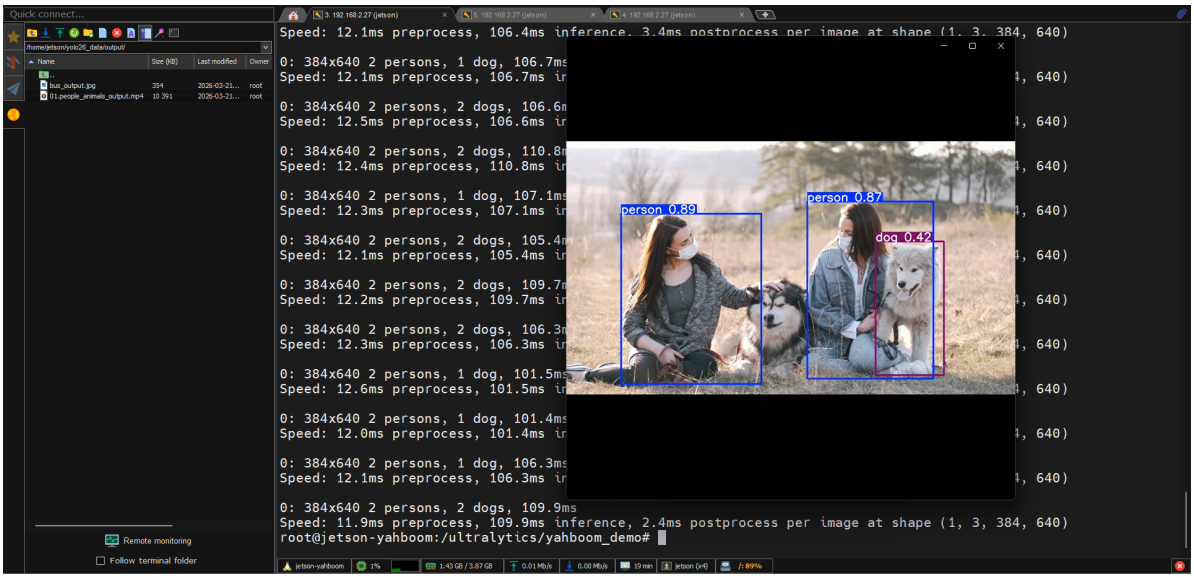
Run the code:

```
python3 01.detection_video.py
```

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

```
root@jetson-yahboom:/ultralytics/yahboom_demo# python3 01.detection_video.py

/ultralytics/ultralytics/nn/modules/head.py:246: UserWarning: __floordiv__ is deprecated, and its behavior will
 change in a future version of pytorch. It currently rounds toward 0 (like the 'trunc' function NOT 'floor'). T
his results in incorrect rounding for negative values. To keep the current behavior, use torch.div(a, b, roundi
ng_mode='trunc'), or for actual floor division, use torch.div(a, b, rounding_mode='floor').
  idx = ori_index[torch.arange(batch_size)[..., None], index // nc]  # original index
0: 384x640 2 persons, 3 dogs, 99.7ms
Speed: 13.2ms preprocess, 99.7ms inference, 2.5ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 dogs, 185.4ms
Speed: 20.6ms preprocess, 185.4ms inference, 3.2ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 dogs, 135.4ms
Speed: 12.4ms preprocess, 135.4ms inference, 2.1ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 dogs, 111.0ms
Speed: 12.0ms preprocess, 111.0ms inference, 3.0ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 dogs, 110.3ms
Speed: 12.0ms preprocess, 110.3ms inference, 3.0ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 dogs, 106.2ms
Speed: 11.9ms preprocess, 106.2ms inference, 2.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 dogs, 106.2ms
Speed: 11.9ms preprocess, 106.2ms inference, 2.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 dogs, 106.3ms
Speed: 12.0ms preprocess, 106.3ms inference, 2.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 dogs, 138.4ms
Speed: 12.6ms preprocess, 138.4ms inference, 2.5ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 dogs, 106.3ms
```



Sample code:

```python
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/ultralytics/yolo26n.pt")

# Open the video file
video_path = "/root/yolo26_data/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))
```

```
# Define the codec and create a VideoWriter object to output the processed video
output_path = "/root/yolo26_data/output/01.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

4. Target Prediction: Real-time Detection (USB Camera)

Use yolo26n.pt for real-time object detection through USB camera.

> Note: Need to connect USB camera first, press `q` key to exit real-time detection

Enter the code folder:

```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 01.detection_camera_usb.py
```

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/ultralytics/yolo26n.pt")


# Open the cammera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter.fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/root/yolo26_data/output/01.detection_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)


        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
```

```
cv2.destroyAllWindows()
```

5. Target Prediction: Real-time Detection (CSI Camera)

Use yolo26n.pt for real-time object detection through CSI camera.

> Note: Please ensure CSI camera is properly connected and use docker startup script that supports CSI camera to enter container

Enter the code folder:

```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 01.detection_camera_csi.py
```

> Press keyboard `q` key to exit real-time detection

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```
import cv2
from ultralytics import YOLO


def gstreamer_pipeline(sensor_id=0, width=640, height=480, fps=30):
    return (
        f"nvarguscamerasrc sensor-id={sensor_id} ! "
        f"video/x-raw(memory:NVMM),width={width},height={height},framerate=
{fps}/1 ! "
        "nvvidconv ! "
        "video/x-raw,format=BGRx ! "
        "videoconvert ! "
        "video/x-raw,format=BGR ! appsink"
    )


# Load the YOLO model
model = YOLO("/ultralytics/yolo26n.pt")


# Open the camera
frame_width = 640
frame_height = 480
fps = 30
cap = cv2.VideoCapture(
    gstreamer_pipeline(width=frame_width, height=frame_height, fps=fps),
    cv2.CAP_GSTREAMER,
)
```

```
if not cap.isOpened():
    raise RuntimeError("Failed to open CSI camera")

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/root/yolo26_data/output/01.detection_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the camera frame cannot be read
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

Reference Materials

Object Detection - Ultralytics YOLO Documentation